A Quorum-Based Total Order Broadcast Protocol for Distributed Intelligence Databases

Distributed Systems, 2025 – 2026

You have been confidentially entrusted by a super secret agency of spies with the development of this project. Due to the sensitive nature of the system, you must not disclose any information regarding the project, especially implementation details or design choices, to anyone outside your assigned project teammate. Sharing information with other course attendees may pose risks to international security and could result in serious consequences. The only authorized channel for asking questions or raising doubts about the **project specification (but not about the solution or implementation)** is email communication with the teaching assistants.

The intelligence services need to track the geographical position of a fixed-size set of people of special interest. Each position is serialized as an `int` value. The positions are therefore stored in an array of integers, with each index corresponding to a specific person of special interest. For security, fault tolerance, and global availability reasons, this list is not stored in a centralized database. Instead, it is maintained by a distributed storage system composed of multiple replicas, geographically distributed worldwide, each storing a copy of the same data. Authorized clients can issue read and write requests to the system by contacting any replica. Your task is to design and implement a distributed protocol for coordinating this group of replicas, ensuring consistent management of the shared list. The system must tolerate multiple node failures through a quorum-based approach and guarantee that all update operations are eventually *applied by all replicas in the same total order* (it is crucial to maintain a consistent chronological order of the persons of special interest's positions). Update requests from external clients are handled through a two-phase total order broadcast protocol, coordinated by a distinguished replica called the *coordinator*. In the event of a coordinator failure, the system must elect a new coordinator to ensure continuity of service.

Background: total order broadcast
Reliable broadcast of a message m satisfies the following properties:

- **Validity**: if the sender is correct, then it eventually delivers m.
- **Integrity**: a process delivers m at most once, and only if it was previously sent.
- **Uniform Agreement**: if a process delivers m, then all correct processes deliver m.
- **Total Order**: if a correct process delivers a message m before a message m', then all correct processes deliver m before m'.

1 Project description

The project must be implemented in Akka with each of the N replicas and client nodes being an Akka actor, identified by a unique ID. Every replica stores a local copy of the list of positions $\mathcal{P}$. External client actors may contact any replica to issue read or write requests. The system provides sequential consistency from each replica's point of view. All operations issued by one client to a specific replica are observed in the same order in which they were sent on the whole system. We assume a static system membership: no new replicas may join after initialization. Replicas, including the coordinator, may crash at any time.

Client requests. Clients may issue read and write requests to any replica. Both request types include the `ActorRef` of the requesting client. For read requests, the client sends the index idx of the value it wants to read. The contacted replica immediately replies with the current value stored at that index in the list (i.e., $\mathcal{P}[idx]$). A write request carries a tuple $\langle idx, val \rangle$ telling the replica to set the value at index idx to val (i.e., $\mathcal{P}[idx] = val$). For write requests, the contacted replica forwards the request to the current coordinator, which is responsible for executing the update protocol.

Update protocol. To process a write request, the coordinator initiates a broadcast-based update protocol. It first sends an `UPDATE` message containing the write request $\langle idx, val \rangle$ to all replicas. Each replica replies with an `ACK` message upon receiving the update. The coordinator waits until it collects acknowledgments from a quorum Q of replicas, where $|Q| = \lfloor N/2 \rfloor + 1$, i.e., a strict majority. Since the coordinator itself is also a replica, the quorum includes the coordinator. Once the quorum is reached, the coordinator broadcasts a `WRITEOK` message. Upon receiving `WRITEOK`, replicas apply the update to their local state. Each update is uniquely identified by a pair $\langle e, i \rangle$, where e denotes the current epoch and i is a sequence number within that epoch. Epoch numbers increase monotonically, and a new epoch is started whenever a new coordinator is elected. The sequence number is reset to 0 at the beginning of each epoch. Replicas maintain a history of updates they have observed, which is later used to recover from coordinator failures (see "Coordinator election").

Crash detection. Both the coordinator and the replicas may fail by crashing. We assume, however, that at all times at least a quorum of $|Q|$ replicas remains correct. Crash detection is based on timeouts. A replica detects that the coordinator has crashed if it does not receive a `WRITEOK` message within a predefined timeout after receiving an `UPDATE`. Similarly, a replica that forwards a write request to the coordinator starts a timeout and detects a failure if the coordinator does not initiate the broadcast protocol in time. To detect silent failures, the coordinator periodically broadcasts `HEARTBEAT` messages, which replicas use to monitor its liveness.

Coordinator election. When a coordinator crash is detected, the remaining replicas elect a new coordinator using a ring-based election protocol. The logical ring is defined by ordering replicas according to their identifiers. `ELECTION` messages circulate along the ring and carry information about the updates known by each replica. When a replica receives an `ELECTION` message, if it is not already participating in the election, it adds its own information about the most recent update it has observed. The message is then forwarded to the next replica in the ring, and the sender is acknowledged with an `ACK`. Replicas may crash during the election process. To tolerate failures, a replica that forwards an `ELECTION` message starts a timeout while waiting for the corresponding `ACK`. If the timeout expires, the sender assumes that the next replica in the ring has crashed, skips it, and forwards the message to the following replica in the ring. Since we assume that a quorum (i.e., a majority) of replicas is always available, at least one correct replica must have knowledge of the most recent update. The elected coordinator is therefore chosen as the replica that knows the most recent update; replica identifiers are used to break ties when multiple replicas are equally up to date. Once elected, the new coordinator broadcasts a `SYNCHRONIZATION` message to announce its leadership. It then brings all other replicas up to date by sending any missing updates, ensuring that all correct replicas converge to the same state. Only after synchronization completes does the coordinator resume processing new write requests.

Properties. The system guarantees the following safety property: if a replica a applies an update w, then all correct (i.e., non-faulty) replicas will eventually apply w, regardless of whether replica a later crashes. This property prevents situations in which a client observes a value that does not exist anywhere else in the system. To preserve this guarantee, a newly elected coordinator must detect and complete any update broadcasts that were interrupted by the previous coordinator's failure. This ensures that partially disseminated updates are either completed or consistently recovered, and that no update acknowledged by a quorum is lost.

2 Implementation-related assumptions and requirements

- The project must be implemented in Akka, with nodes being Akka actors, written in Java programming language. In exceptional circumnstances, the project may be implemented using alternative frameworks or languages; however, **this must first be discussed with the instructors**.
- You will be given a codebase to start developing your project. The codebase is available at https://github.com/StefanoGenettiUniTN/ds-student-project-2026. It is **mandatory** to use it, read the documentation, and adhere to the guidelines.
- The codebase will provide some basic automated tests. Submissions that do not pass ALL tests will not be considered. Not adhering to the codebase requirements will make tests fail.
- Communication channels are assumed to be reliable and FIFO.
- To emulate network propagation delays, random (but controlled) delays must be introduced between message transmissions, as demonstrated in the examples discussed in class. Please follow the related guidelines in the codebase carefully.
- A strict majority of replicas never crashes.
- Replicas fail by crashing and do not recover.
- Proper actor encapsulation must be ensured. Shared mutable state is forbidden; any shared objects must be immutable.
- The program must log the key steps of the protocol, for instance:
  - `[Client <ClientName>] requesting READ (<idx>) to <ReplicaID>`
  - `[Client <ClientName>] requesting WRITE (<idx>, <val>) to <ReplicaID>`
  - `[Client <ClientName>] READ complete (<isSuccess>, <idx>, <val>) from <ReplicaID>`
  - `[Client <ClientName>] WRITE complete (<isSuccess>, <idx>, <val>) from <ReplicaID>`
  - `[Client <ClientName>] TIMEOUT READ request to <ReplicaID> (<idx>)`
  - `[Client <ClientName>] TIMEOUT WRITE request to <ReplicaID> (<idx>, <val>)`
  - `[Replica <ReplicaID>] applied update <epoch>:<sequence number> (<idx>, <val>)`
  - `[Replica <ReplicaID>] CRASHED`

  All logs must first show the timestamp with millisecond precision. The codebase will provide documented utility functions to print such logs (some logs are already implemented). You must use them because, if prints are enabled during automated tests, they can interfere with their outcome (even if the implementation is correct).
- To support automated testing, the system should allow the execution of a sequence of write operations (possibly interleaved with crash events) while a client repeatedly performs read operations on a replica.

- To emulate crashes, a replica must be able to enter a *crashed mode*, in which it ignores all incoming messages and stops sending messages to other actors. Please follow the guidelines in the codebase.
- Crash detection is assumed to be accurate and does not produce false positives. Timeout values should therefore be chosen reasonably.
- For evaluation purposes, it must be possible through minimal instrumentation of the code to trigger crashes at specific points in the protocol execution, such as during the broadcast of an `UPDATE`, after receiving an `UPDATE`, during the dissemination of `WRITEOK` messages, or while the coordinator election is in progress. You will find more details in the codebase. Again, follow the guidelines.
- It is important to state all additional assumptions in the report.

3 Project report

You are required to submit a **short report of 3–4 pages (max. 6 pages). Reports exceeding this page limit will be automatically rejected, and the project will need to be resubmitted**. The report must be written in English using the provided LaTeX template and should clearly describe the main architectural and design choices of your implementation. The report should explain how the proposed design and implementation satisfy the project requirements; to guide your writing, refer to the project presentation slides, which include a list of example questions that your report is expected to address.

The report must end with a short paragraph titled "**Information about the use of LLM tools**" containing, for each LLM tool used, its name and the purpose of its use (e.g., checking text already written, generating it from scratch, creating code, etc.). If no LLM has been used, simply state so. Whether LLMs have been used or not does not affect the evaluation of your project that, as discussed next, will be based on the technical content of the project and your ability to illustrate it effectively and answer questions about it. However, **this section is mandatory**: you cannot present your project unless you have provided the above information.

4 Presenting the project

You are responsible for demonstrating that your project works correctly. The project will be evaluated primarily on its technical content, with particular emphasis on the correctness of the implemented algorithms. *Do not* spend time implementing features beyond those explicitly required; instead, focus on implementing the core functionality accurately.

The project presentation consists of two parts. In the first part, you will have **12 minutes** to present your solution. A timer will be used, and exceeding the allotted time will not be allowed. After the presentation, in the second part, you will be asked additional questions about the project. You must bring your laptop with a working version of the code. During the discussion, you may be asked to show parts of the implementation, execute the program, explain design choices, and describe how specific features are implemented or how the system could be extended. We strongly recommend preparing three or four representative execution examples, including corner cases, to demonstrate the correctness and robustness of your implementation.

A correct implementation of all the requested functionality is worth 6 points. Submissions that implement only a subset of the required features, or that rely on stronger assumptions than those specified, may still be accepted but will receive a lower mark.

The project must be developed by exactly two students. However, grading is individual and will be based on each student's understanding of the code and of the underlying concepts.

- Register your group by filling out this Google Form. If you missed it, do not worry, you will still be able to present, but it is strongly encouraged as it helps with internal organization.
- To book your project presentation, find an empty slot in this shared GoogleSheet. Here you will find available presentation slots along with the corresponding submission deadlines. To reserve a slot, you **must** contact both the instructor (gianpietro.picco@unitn.it) and the teaching assistants (thomas.pasquali@unitn.it, stefano.genetti@unitn.it) by email *before* the corresponding submission deadline, and submit your project.
- Submit the code and report by email. The report must be a single PDF file. Place all source files and the report in one directory named after the students' surnames (e.g., RossiRusso), then compress the directory into a .tgz archive using `tar + gz` (e.g., `tar -czvf RossiRusso.tgz RossiRusso`).
- The project may be presented at any time within the available slots, even outside official exam sessions. In such cases, the mark will be recorded but will remain "frozen" until you enroll in the next exam session.
- Code must be properly formatted; submissions that do not follow reasonable standards will not be accepted.
- The project will be demonstrated in person or online in front of the instructor and/or a teaching assistant.
- Before sending an email with questions or doubts regarding the **project specification (but not the solution or implementation)**, please first consult the shared FAQ document.
- Project deadline: expected around May 2027 (next edition's project presentation). If you have started but cannot meet this date, contact the instructor.
- **Plagiarism is not tolerated.** If you encounter difficulties during the implementation, do not hesitate to ask questions. We are here to help.